

Guía de Estudio: Fundamentos de Ingeniería de Software y el Proyecto DeV4GiRLS

Esta guía de estudio tiene como objetivo revisar y consolidar los conceptos fundamentales presentados en los materiales del proyecto DeV4GiRLS. El enfoque se centra en la transición del pensamiento de programación básica (Scratch) hacia una mentalidad de ingeniería de software, el diseño de sistemas y el uso crítico de la inteligencia artificial.

I. Cuestionario de Repaso

Responda a las siguientes preguntas de forma concisa (2 a 3 oraciones cada una).

1. ¿Cuál es la diferencia fundamental entre una "app" y un "juego" según el marco de este proyecto?
2. En la comparación con Scratch, ¿qué significa pasar de un "escenario mezclado" a "partes separadas y organizadas"?
3. ¿Cuál es la analogía de la mochila y qué concepto de ingeniería intenta explicar?
4. ¿Por qué se enfatiza el diseño con "cajas y flechas" antes de escribir cualquier línea de código?
5. ¿Qué funciones específicas cumple el "Sistema de Navegación" dentro de la arquitectura de la aplicación?
6. ¿Bajo qué criterio se considera que el diseño de una aplicación es "bueno" al intentar añadir nuevas funcionalidades?
7. ¿Cuál es la razón pedagógica para prohibir el uso de IA durante la primera sesión de diseño?
8. Según el documento del Proyecto Educativo, ¿cuál es el rol definitivo que debe ocupar la IA frente al estudiante?
9. ¿Por qué el éxito del proyecto no se mide únicamente por la publicación de la aplicación en una tienda?
10. ¿Qué diferencia existe entre la "responsabilidad" de un juego individual y la del "menú principal"?

II. Clave de Respuestas

1. **Diferencia entre App y Juego:** Un juego es una actividad específica con reglas propias (como el Tic Tac Toe), mientras que una app es el programa completo que contiene y organiza varios de estos juegos dentro de un sistema común.

2. **Escenarios vs. Partes Organizadas:** Significa evolucionar de proyectos donde todo el código y los elementos visuales están en un mismo sitio, hacia un sistema modular donde cada componente tiene una función delimitada y está separado de los demás.
3. **Analogía de la Mochila:** Se utiliza para ilustrar que un sistema complejo no es un "objeto gigante", sino una unión de partes con responsabilidades distintas (cremalleras, bolsillos) que juntas forman algo útil.
4. **Diseño antes del Código:** Se busca que la alumna decida la estructura y el flujo de la aplicación (pensamiento y diseño) para evitar sistemas caóticos que surgen al programar de forma improvisada.
5. **Sistema de Navegación:** Su responsabilidad es permitir el cambio fluido entre el menú principal y los diferentes juegos, además de gestionar el retorno al menú sin interferir en las reglas propias de cada juego.
6. **Criterio de un Buen Diseño:** Un diseño es considerado de alta calidad si permite añadir una nueva pieza (como un cuarto juego) de manera independiente, sin necesidad de rehacer o modificar profundamente el resto del sistema existente.
7. **Restricción Inicial de la IA:** Se evita su uso al principio porque si se genera código antes de entender el diseño, se obtendrán resultados funcionales que la alumna no comprenderá, perdiendo el objetivo de aprendizaje.
8. **Rol de la IA:** La IA es definida como un "ayudante" o "copiloto" imperfecto que puede sugerir o explicar código, pero nunca debe sustituir al "arquitecto" (el estudiante), quien mantiene el control del diseño y las decisiones.
9. **Medición del Éxito:** El éxito se define por la capacidad de la alumna para explicar el funcionamiento del sistema, razonar sobre sus decisiones técnicas y mantener un criterio propio, más allá de la funcionalidad del producto final.
10. **Responsabilidades:** El menú principal se encarga de mostrar opciones y permitir la elección entre juegos, mientras que cada juego es responsable exclusivamente de sus propias reglas, sin necesidad de saber que otros juegos existen.

III. Temas para Desarrollo de Ensayos

Proponga respuestas extensas y argumentadas para los siguientes temas (no se incluyen respuestas):

1. **La importancia de la arquitectura en el software:** Analice por qué la separación de responsabilidades y la modularidad son vitales para transformar un simple programa en un sistema de ingeniería escalable.
2. **Del bloque al sistema:** Reflexione sobre cómo los conceptos aprendidos en Scratch (eventos, lógica de estados) sirven de base, pero resultan insuficientes para la creación de aplicaciones profesionales sin una estructura de diseño previa.
3. **Ética y metodología en el uso de IA generativa:** Discuta las consecuencias de delegar el diseño arquitectónico a una IA y cómo el principio de "comprender antes de ejecutar" protege el proceso educativo.

4. **El concepto de "Módulos" en el desarrollo de aplicaciones:** Explique cómo la visión de los juegos como módulos independientes facilita el mantenimiento de una aplicación y la colaboración entre diferentes partes del sistema.
5. **Pensamiento Crítico y Resolución de Problemas:** Evalúe cómo el planteamiento de preguntas sobre posibles fallos o cambios (bugs, cambios de color, escalabilidad) fomenta una mentalidad de ingeniería en lugar de una simple habilidad técnica.

IV. Glosario de Términos Clave

Término	Definición según el Contexto del Proyecto
Agente de IA	Herramienta de inteligencia artificial utilizada como copiloto para explicar fragmentos de código, sugerir refactorizaciones o proponer alternativas, siempre bajo supervisión humana.
API	Interfaz de programación que permite la comunicación entre diferentes componentes de software (concepto a introducir a nivel conceptual).
App (Aplicación)	El programa completo e integral que se instala en un dispositivo, compuesto por un sistema que alberga múltiples funciones o juegos.
Arquitectura	La estructura interna del software que define cómo se organizan, relacionan y comunican las diferentes partes de un sistema.
Estado	Situación específica en la que se encuentra el programa en un momento dado (ej. "jugando", "ganado", "perdido").
Ingeniería de Software	Disciplina que aplica principios de diseño y organización para construir sistemas de software complejos, mantenibles y escalables.
Juego (Módulo)	Actividad con reglas propias que, en este proyecto, funciona como una pieza independiente dentro del sistema general de la aplicación.

Navegación	Mecanismo o flujo que permite al usuario moverse entre las diferentes pantallas o secciones de la aplicación (del menú al juego y viceversa).
Responsabilidad	La tarea o función única que tiene asignada una parte específica del sistema (ej. el menú solo muestra opciones, no juega).
Sistema	Conjunto de partes organizadas que colaboran entre sí para cumplir un objetivo común, siguiendo reglas de estructura y jerarquía.